

Unidad 3

Programación de comunicaciones

3.2 TCP

Programación de servicios y procesos

Curso 2025/2026

Contenido

- ▶ Clase InetAddress
- ▶ Clases Socket y ServerSocket
- ▶ Conexión y desconexión
- ▶ Transmisión y recepción en TCP
 - ▶ I/O Streams

Clase InetAddress

- ▶ Representa una dirección IP (versión 4 o 6)
- ▶ Se usa para:
 - ▶ Resolver nombres DNS → IP
 - ▶ Pasar direcciones a los sockets
 - ▶ Obtener información básica del host
- ▶ **Los sockets trabajan con InetAddress**

Como se construye InetAddress

```
// getByName - construye un nombre de host  
InetAddress addr = InetAddress.getByName("www.google.com");
```

```
// getLocalHost - obtiene la dirección local  
InetAddress local = InetAddress.getLocalHost();
```

```
// getAllByName: obtiene todas las direcciones IP asociadas a un nombre  
InetAddress[] addrs = InetAddress.getAllByName("www.google.com");
```

```
// getByAddress: crea una dirección a partir de bytes (sin DNS)
```

```
byte[] ip = {(byte)192, (byte)168, 1, 10};  
InetAddress addr = InetAddress.getByAddress(ip);
```

```
// aunque se suele usar:
```

```
InetAddress addr = InetAddress.getByName("192.168.1.10");
```

Métodos de InetAddress

- ▶ Convertir a string

```
addr.getHostAddress(); // "142.250.184.36"  
addr.getHostName();    // "www.google.com"  
addr.toString();        // "www.google.com/142.250.184.36"
```

- ▶ Alcanzabilidad (ping)

```
addr.isReachable(2000); // timeout en ms
```

Clases Serversocket y Socket

- ▶ Incluidas en java.net
- ▶ Clase **ServerSocket**
 - ▶ Se usa en el lado servidor
 - ▶ Escucha peticiones
 - ▶ No envía ni recibe datos
- ▶ Clase **Socket**
 - ▶ En el lado cliente: inicia conexiones
 - ▶ En ambos lados: envía y recibe datos

¿Cómo funciona? - 1) Constructores

▶ Servidor

- ▶ Crea un ServerSocket
- ▶ Su función es escuchar en un puerto
- ▶ No sirve para enviar ni recibir datos

```
ServerSocket serverSocket = new ServerSocket(5000);
```

▶ Cliente

- ▶ Crea directamente un Socket
- ▶ Indica host + puerto
- ▶ Con esto se inicia la conexión

```
Socket socket = new Socket("localhost", 5000);
```

¿Cómo funciona? - 2) accept()

- ▶ En el servidor:

```
Socket socket = serverSocket.accept();
```

- ▶ accept() bloquea hasta que un cliente se conecta, y entonces:
 - ▶ Se establece la conexión TCP
 - ▶ Se devuelve un **nuevo Socket**
 - ▶ Ese Socket representa **esa conexión concreta con ese cliente**
- ▶ El ServerSocket **no desaparece**:
 - ▶ Puede seguir aceptando más clientes
 - ▶ Cada cliente → su propio Socket

¿Cómo funciona?

3) Obtener streams de los sockets

- ▶ Una vez hay un socket en cada extremo se obtienen streams
- ▶ En ambos lados:

```
InputStream in = socket.getInputStream();  
OutputStream out = socket.getOutputStream();
```

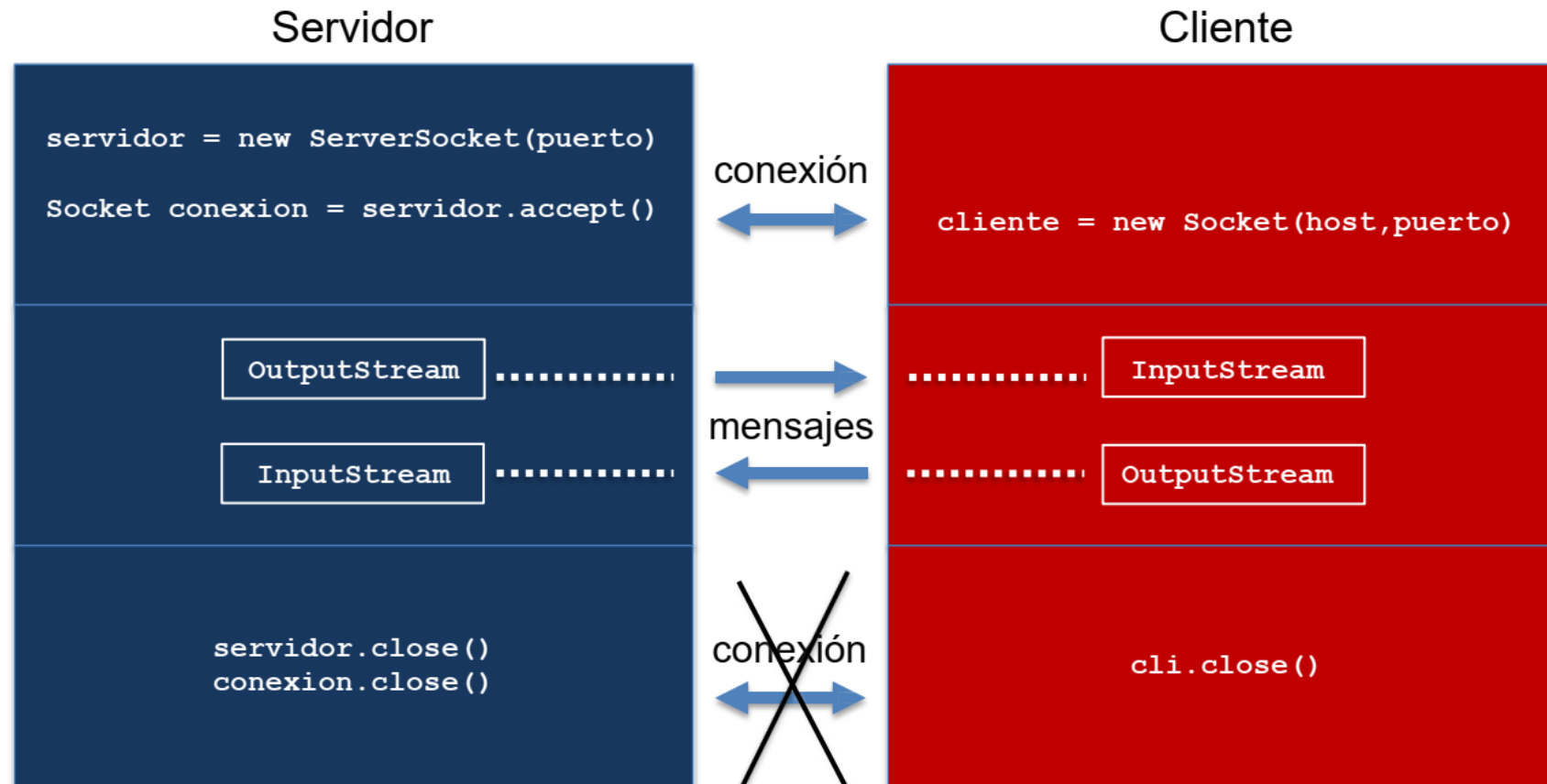
- ▶ Ya tenemos un canal bidireccional
- ▶ Los streams se pueden convertir a:
 - ▶ BufferedReader / PrintWriter para texto
 - ▶ DataInputStream / DataOutputStream para int, double...
 - ▶ ObjectInputStream / ObjectOutputStream para objetos serializados
 - ▶ Importante **no mezclar tipos de stream** - usar del mismo tipo a ambos lados

¿Cómo funciona?

4) Cerrar

- ▶ El orden importa
 1. Streams
 2. Socket
 3. ServerSocket
- ▶ Si el cliente cierra, el servidor no recibe un evento de cierre
- ▶ Lo sabe leyendo del stream
 - ▶ Con bytes: -1
 - ▶ Con texto: null
 - ▶ O bien una excepción si el cliente se ha cerrado mal
 - ▶ SocketException, EOFException, IOException

En resumen...



Clase ServerSocket - métodos

Métodos	Devuelve	Uso
ServerSocket (int puerto)	-	Constructor. Crea un socket de servidor que se enlaza al puerto
accept()	Socket	Espera hasta que reciba una solicitud de conexión de un cliente Devuelve un Socket para comunicaciones
close()	-	Cierra el ServerSocket
getLocalPort()	int	Devuelve el puerto local al que está enlazado el ServerSocket

Clase Socket - métodos

Método	Devuelve	Uso
Socket(Host, puerto)	-	Constructor. Crea un socket y lo conecta al host y puerto especificado
getInputStream()	InputStream	Devuelve un InputStream para leer bytes del socket
getOutputStream()	OutputStream	Devuelve el OutputStream que permite escribir bytes sobre el socket
close()	-	Cierra el socket
getInetAddress()		Devuelve la dirección IP a la que está conectado el socket
getLocalPort	int	Devuelve el puerto local al que está conectado el socket
getPort	int	Devuelve el puerto remoto al que está conectado el socket

Código en el servidor

```
import java.io.* ;
import java.net.* ;

class Servidor {
    static final int Puerto = 2000;
    public Servidor( ) {
        try {
            // Inicio la escucha del servidor en un determinado puerto
            ServerSocket skServidor = new ServerSocket(Puerto);
            System.out.println("Escuchando el puerto " + Puerto );
            // Espero a que se conecte un cliente y creo un nuevo socket
            para el cliente
            Socket sCliente = skServidor.accept();
            // Aquí atiendo la petición del cliente ...
            // Cierro el socket
            sCliente.close();
        } catch( Exception e ) {
            System.out.println( e.getMessage() );
        }
    }
    public static void main( String[] arg ) {
        new Servidor();
    }
}
```

Código del cliente

```
import java.net.Socket;

class Cliente {
    static final String Host = "localhost";
    static final int Puerto = 2000;

    public Cliente() {
        try {
            // Me conecto al servidor en un determinado puerto
            Socket sCliente = new Socket(Host, Puerto);
            // TAREAS QUE REALIZA EL CLIENTE
            // Cierro el socket
            sCliente.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] arg) {
        new Cliente();
    }
}
```

Actividad 1 - Conexión

- ▶ A partir de los esqueletos proporcionados:
 - ▶ En el cliente, indicar el puerto local y el puerto remoto de la conexión, así como la dirección IP
 - ▶ En el servidor. con el mismo Serversocket esperar y cerrar la conexión de tres clientes consecutivos de forma secuencial indicando el número de cliente que se ha conectado
- ▶ Para evitar pelearnos con firwalls, ejecutaremos los servidores y clientes en local

Recordatorio I/O Streams

- ▶ Streams de bytes
 - ▶ Para datos binarios: imágenes, audio, ficheros, **sockets**, etc.
 - ▶ Clases base:
 - ▶ **InputStream**
 - ▶ **OutputStream**
- ▶ Streams de caracteres
 - ▶ Para texto
 - ▶ Clases base
 - ▶ **Reader**
 - ▶ **Writer**

Recordatorio: I/O Streams de bytes

- ▶ Para leer bytes: **InputStream**
 - ▶ Clases derivadas:
 - ▶ **BufferedInputStream** - añade buffer
 - ▶ **ObjectInputStream** - lee objetos serializados
 - ▶ **DataInputStream** - lee tipos primitivos (int, double...)
- ▶ Para escribir bytes: **OutputStream**
 - ▶ Clases derivadas
 - ▶ **BufferedOutputStream**
 - ▶ **ObjectOutputStream**
 - ▶ **DataOutputStream**

Recordatorio: I/O streams de caracteres

- ▶ Para leer caracteres: **Reader**
 - ▶ **BufferedReader**: leer líneas (`readLine()`)
 - ▶ **InputStreamReader**: puente bytes → caracteres
- ▶ Para escribir caracteres
 - ▶ **PrintWriter** → escritura con líneas (`println`)
 - ▶ Se puede usar **BufferedWriter** con `write` - obliga a poner saltos de línea y hacer `flush()`
 - ▶ **OutputStreamWriter**: puente caracteres → bytes

Buenas práctica con sockets

- ▶ Siempre usar el mismo tipo de streams al otro lado
 - ▶ No es que no se puedan usar streams distintos, pero es un lío
- ▶ En las clases puente es una buena idea indicar la codificación si estamos con sistemas distintos:
 - ▶ Ejemplo:

```
ir = new InputStreamReader(sCliente.getInputStream(), StandardCharsets.UTF_8));  
ow = new OutputStreamWriter(sCliente.getOutputStream(), StandardCharsets.UTF_8)
```

- ▶ Si no, se usa la codificación por defecto - puede variar entre sistemas

Actividad 2 - Comunicación

- ▶ Modificar el resultado de la actividad 1 para que se intercambien información textual
 - ▶ El servidor
 - ▶ enviará el texto “Hola cliente #” con el número de cliente
 - ▶ Leerá lo que le haya enviado el cliente mostrándolo precedido del texto “Mensaje del cliente : “
 - ▶ El cliente
 - ▶ Leerá lo que le haya enviado el servidor, mostrándolo por pantalla precedido del texto: “Mensaje del servidor”
 - ▶ Pedirá un mensaje en el usuario para enviar al servidor.